ShowUp: A Relational Database–Driven Campus Event Discovery, Registration and Check-in System

Rayna Agarwal
Registration No. 25BCE0703
Department of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Faculty Guide: Prof. Pooja Chaudhary
Database Management Systems — Course Project Report · Source code: github.com/raynaagarwal01-rgb/ShowUp

***Abstract*—Campus events are still registered through a patchwork of Google Forms, spreadsheets and messaging groups, which produces duplicated data, over-subscribed events, lost team codes and paper attendance. This report presents ShowUp, a web application for discovering events across colleges, registering solo or as a team, receiving a QR ticket and being checked in at the venue. The work follows the complete database development life cycle: requirement analysis; an Entity–Relationship (ER) model with an Enhanced ER (EER) specialization; mapping to a relational schema verified to be in Third Normal Form (3NF) and Boyce–Codd Normal Form (BCNF); implementation in MySQL 8.0 as eight InnoDB tables with twelve foreign keys and declarative CHECK, UNIQUE and NOT NULL constraints; and data population. A React front end communicates with a Node.js/Express tier that issues only parameterised SQL and enforces deadlines, capacity, duplicate prevention and team-size limits inside ACID transactions with row-level locks. Thirty-five test cases executed against a live MySQL 8.0.46 server all passed, including five simultaneous registrations for a single remaining seat.**

***Index Terms*—Database design, ER model, EER specialization, normalization, MySQL, SQL, transactions, concurrency control, web application.**

I. Introduction

Universities host many co-curricular events each semester. At VIT Vellore a student may in one week be invited to a hackathon, a workshop, an e-sports tournament and a guest lecture, each organised by a different club with its own registration form. Organisers export responses to spreadsheets, reconcile duplicates, chase incomplete teams and mark attendance on paper.

These are *data management* problems: the same student is stored many times in unrelated files, there is no authoritative count of seats taken, team membership is free text, and nothing prevents two people from taking the last seat at once. The relational model [3] and an RDBMS are designed precisely to remove such redundancy and to enforce integrity under concurrent access [1], [2]. ShowUp replaces the forms with one platform backed by a normalized MySQL database: students browse events from several colleges, register individually or form a team that friends join with a six-character code, and receive a QR ticket; organisers manage events, view participants by team, post announcements, answer questions, publish results and check participants in.

The objectives are to (i) analyse the requirements, (ii) design ER/EER models and map them to relations, (iii) normalize every relation to at least 3NF, (iv) implement the schema with SQL DDL and DML, (v) build a front end and an application tier that enforces business rules transactionally, and (vi) validate the system with documented tests. Payments, e-mail/SMS notifications and waitlists are out of scope. Sections II–XII follow the eleven project stages in order; Section XIII concludes.

II. Problem Identification and Requirement Analysis

A. Problem Statement

Design and implement a database-driven system that lets students of multiple colleges discover events and register individually or as teams, and lets organisers manage events and attendance, while guaranteeing that (i) no student is registered twice for an event, (ii) no event exceeds its capacity even under simultaneous requests, (iii) registrations close at the deadline, (iv) teams respect minimum and maximum sizes, and (v) every fact is stored exactly once.

B. Analysis of the Existing Process

An analysis of the Google Forms, spreadsheets and messaging-group workflows that clubs typically use revealed deficiencies that each map to a database concept:

- **Redundancy and update anomalies**—a student's details are retyped on every form; correcting one copy leaves others stale.
- **No integrity constraints**—forms accept duplicates, invalid team sizes and late registrations.
- **No concurrency control**—capacity is checked manually after the fact, so events are over-subscribed.
- **Repeating groups**—team members are typed into one cell and cannot be queried or validated.

C. Stakeholders and Requirements

Three actors were identified: a *student* registers, forms or joins teams and asks questions; an *organizer* (club account) manages its own events and checks participants in; a *visitor* may only view the landing page and sign up. Following IEEE Std 830 [6], requirements are numbered so they can be traced to design and tests.

TABLE I
Functional Requirements

ID	Requirement	Actor
FR-1	Sign up / log in; role student or organizer	All
FR-2	List events with organizer, college, seats left; search and filter	Student
FR-3	Register for a solo event before the deadline if seats remain	Student

ID	Requirement	Actor
FR-4	Create a team (unique join code) or join one with its code	Student
FR-5	Cancel a registration, leaving the team and freeing the seat	Student
FR-6	View own registrations with QR ticket and team	Student
FR-7	Create, edit and delete own events	Organizer
FR-8	View participants by team; export CSV	Organizer
FR-9	Check participants in by scanning or typing the ticket	Organizer
FR-10	Post announcements, answer questions, publish winners	Organizer

Non-functional requirements are: *integrity*—keys and constraints enforced by the DBMS, not only by code; *consistency*—multi-statement operations are atomic; *concurrency*—capacity can never be exceeded; *security*—bcrypt password hashes, parameterised SQL and role-based authorization; and *portability*—any OS with Node.js 20+ and MySQL 8.0.16+. The data to be stored comprises users, events, teams, team membership, registrations (with status and check-in time), announcements, questions with optional answers, and winners. Seats left is derived and not stored. The software stack is MySQL 8.0 (InnoDB), Node.js with Express 5 [8] and mysql2, and React 19 with TypeScript [9].

III. Construction of ER/EER Diagram

The conceptual model uses Chen's notation [4] with the EER extensions of [1]. Nouns in the requirements became entities, verbs became relationships and descriptive nouns became attributes. Fig. 2 shows the complete diagram; Table II lists the relationships.

TABLE II
Relationship Sets

Relationship	Entities	Card.	Total side
ORGANIZES	USER – EVENT	1:N	EVENT
REGISTRATION	USER – EVENT	M:N	—
PART_OF	REGISTRATION – TEAM	N:1	—
MEMBER_OF	USER – TEAM	M:N	TEAM
LEADS / FORMS	USER / EVENT – TEAM	1:N	TEAM
POSTS / HAS_RESULT	EVENT – ANNOUNCEMENT / WINNER	1:N	child
ASKS / ABOUT	USER / EVENT – QUESTION	1:N	QUESTION

REGISTRATION is an *associative entity*: it carries its own attributes (status, check-in time, timestamp), takes part in PART_OF with TEAM, and its identifier is printed in the QR ticket (`SHOWUP-REG-<id>`). MEMBER_OF carries *joined_at*, used to choose a new leader when the leader leaves. Design assumptions: capacity counts people, so each team member holds an individual registration; a join code is unique site-wide while a team name is unique only within its event; and a cancelled registration is retained as history.

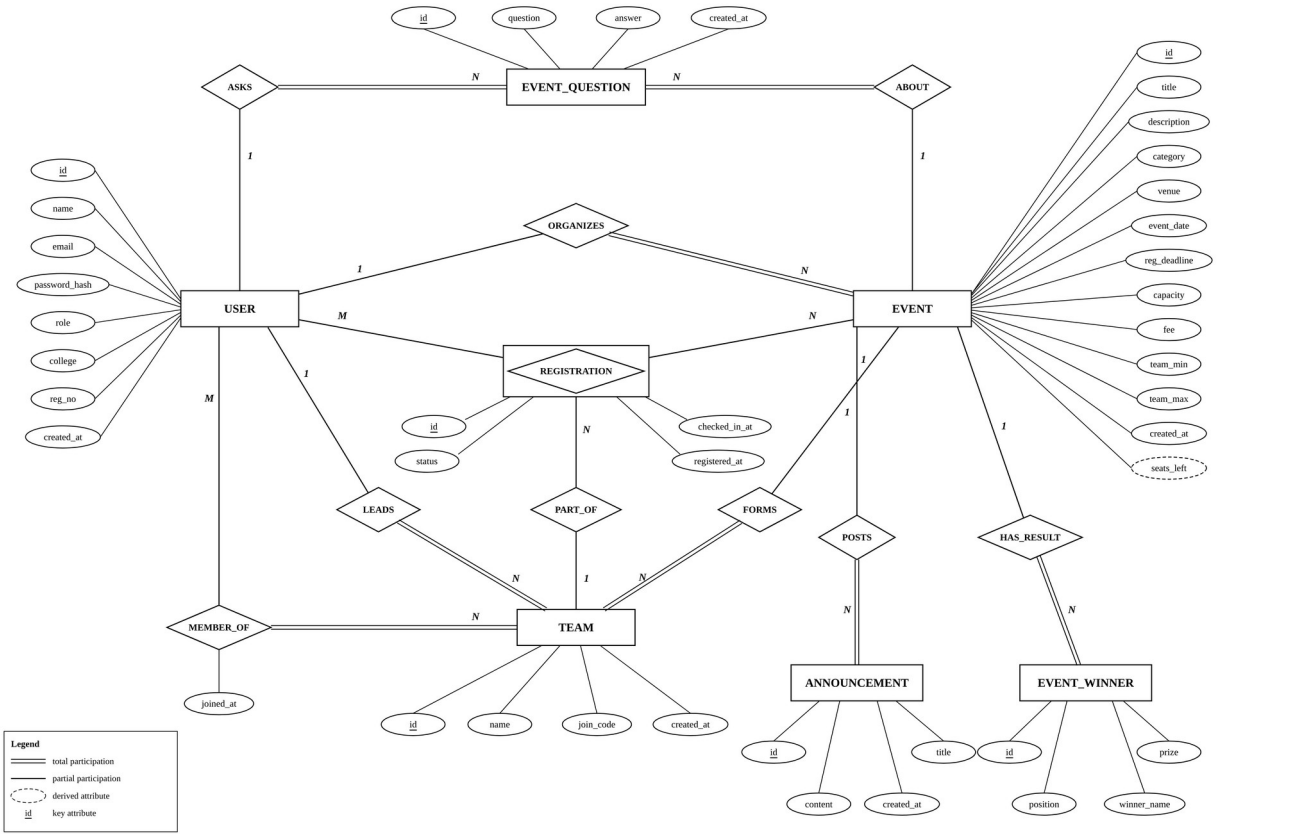


Fig. 2. ER diagram of ShowUp in Chen notation: seven entity sets (REGISTRATION is associative), ten relationship sets, cardinality ratios and participation constraints. The dashed ellipse is the derived attribute seats_left.

Students and organizers share most attributes but take part in different relationships, so Fig. 3 specializes USER into STUDENT and ORGANIZER on the defining attribute *role*. The specialization is **disjoint** and **total**; STUDENT has the specific attribute *reg_no*.

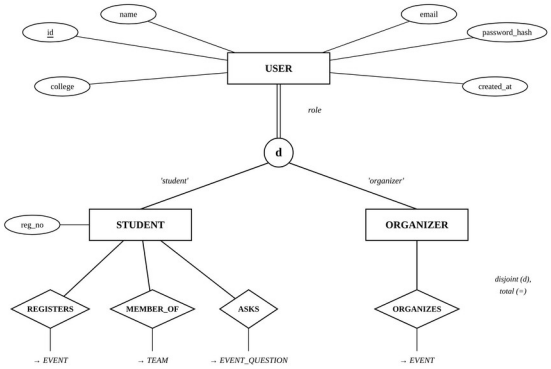


Fig. 3. EER specialization of USER (disjoint, total, attribute-defined on role).

IV. ER Diagram to Relational Model

The ER-to-relational mapping algorithm of [1] was applied (Table III). Surrogate keys (`INT AUTO_INCREMENT`) are used for every entity because natural keys such as e-mail may change and would widen foreign keys.

TABLE III
Application of the Mapping Algorithm

Step	Rule	Result
1	Strong entity → relation	users, events, teams, announcements, event_questions, event_winners
2, 3, 6, 7	Weak entity, 1:1, multivalued, n-ary	none present
4	1:N → FK on N side	organizer_id, event_id, leader_id, user_id, team_id
5	M:N → new relation	team_members (composite PK); registrations with UNIQUE(user_id, event_id)
8	Specialization	Option 8C: one relation users with type attribute role

Option 8C was chosen because the subclasses have only one specific attribute, the specialization is disjoint, and log-in must find a user by e-mail regardless of role; role-specific participation is enforced in the application tier. The resulting schema (primary keys underlined, foreign keys italic and marked †) is:

users(id, name, email, password_hash, role, college, reg_no, created_at)
events(id, *organizer_id*†, title, description, category, venue, event_date, registration_deadline, capacity, fee, team_min, team_max, created_at)
teams(id, *event_id*†, name, join_code, *leader_id*†, created_at)
team_members(*team_id*†, *user_id*†, joined_at)
registrations(id, *user_id*†, *event_id*†, *team_id*†, status, checked_in_at, registered_at)
announcements(id, *event_id*†, title, content, created_at)
event_questions(id, *event_id*†, *user_id*†, question, answer, created_at)
event_winners(id, *event_id*†, position, winner_name, prize)

Fig. 4 shows the twelve foreign keys. All use ON DELETE CASCADE, expressing existence dependency—deleting an event removes its teams, memberships, registrations, announcements, questions and winners in one statement—except `registrations.team_id`, which uses SET NULL so a registration can outlive its team.

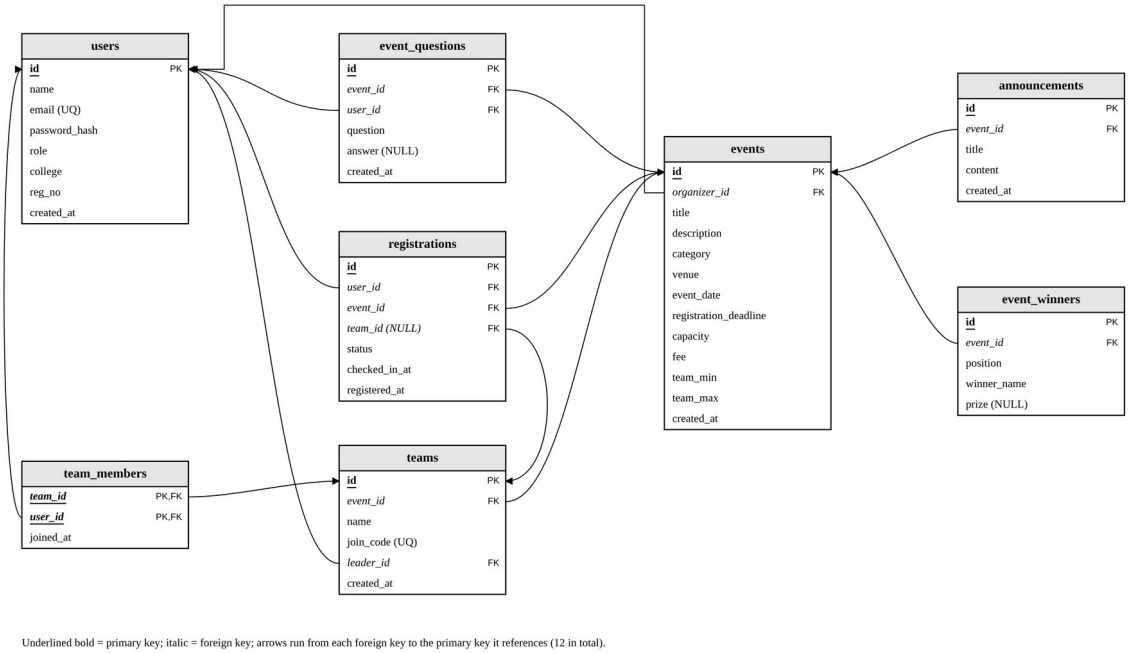


Fig. 4. Relational schema diagram; each arrow is a FOREIGN KEY from the referencing column to the referenced primary key.

V. Normalization (1NF, 2NF, 3NF)

The flat sign-up sheet used by clubs was normalized step by step [1], [5]; this independently yields the relations obtained from the ER mapping.

Unnormalized form: one row per team with members in one cell—`Sheet(event_title, event_date, venue, organizer_name, organizer_college, team_name, join_code, {member_name, email, reg_no, college, status, checked_in})`. The braces mark a repeating group.

1NF: the group is flattened to one row per (event, member) with key (event_id, member_email); per-event repeating groups (announcements, questions, winners) become separate relations with event_id as a foreign key.

2NF: partial dependencies are removed:

```
event_id -> title, date, venue, organizer_email,
            organizer_name, organizer_college
email    -> member_name, reg_no, member_college
event_id, email -> team_name, join_code,
            status, checked_in            (full)
```

giving EVENT(event_id, …), MEMBER(email, …) and SIGNUP(event_id, email, team_name, join_code, status, checked_in); the decomposition is lossless because each shared attribute is a key of one projection.

3NF: transitive dependencies are removed:

```
event_id -> organizer_email -> organizer_name,
                               organizer_college
event_id, email -> team_name -> join_code, leader
```

Organizers are users, so EVENT keeps only `organizer_id` and the organizer's college is obtained by a JOIN—the change made when event-level copies of the host college were dropped during the project's simplification. TEAM(id, event_id, name, join_code, leader_id) and TEAM_MEMBERS(team_id, user_id) are extracted, SIGNUP becomes REGISTRATIONS, and MEMBER and organizer data merge into USERS with *role*, matching the EER model.

Verification: Table IV lists the FDs of the final relations. Every determinant is a candidate key, so every relation is in BCNF (hence 3NF and 2NF).

TABLE IV
Functional Dependencies of the Final Schema

Relation	Candidate keys (each → all attributes)	NF
users	id; email	BCNF
teams	id; join_code; (event_id, name)	BCNF
team_members	(team_id, user_id) → joined_at	BCNF
registrations	id; (user_id, event_id)	BCNF
events, announcements, event_questions, event_winners	id	BCNF

Seats left is not stored but computed as `capacity − COUNT(*)` of confirmed registrations, avoiding an update anomaly. The pair (user, team) appears both in `team_members` and as `registrations.team_id`; this inter-relational redundancy is not a normal-form violation, and it is controlled by always writing both rows in one transaction (Section IX).

VI. Database Design and Table Creation (DDL)

Listing 1 is the DDL in `sql/schema.sql`. Tables are created parent-first so every foreign key references an existing table, and `CREATE TABLE IF NOT EXISTS` makes the script idempotent. InnoDB provides foreign keys, transactions and row-level locks, and indexes every key and UNIQUE column automatically; MySQL 8.0.16+ is required because earlier versions ignore CHECK constraints [7]. The structure was verified with `SHOW TABLES`, `DESCRIBE` and `information_schema.REFERENTIAL_CONSTRAINTS`.

Listing 1. DDL for the ShowUp database (sql/schema.sql, comments omitted).

```
CREATE DATABASE IF NOT EXISTS showup
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE showup;
CREATE TABLE IF NOT EXISTS users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  email         VARCHAR(255) NOT NULL UNIQUE,
```

```
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('student','organizer') NOT NULL
               DEFAULT 'student',
  college       VARCHAR(150) NULL,
  reg_no        VARCHAR(30)  NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS events (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  organizer_id INT          NOT NULL,
  title        VARCHAR(200) NOT NULL,
  description  TEXT         NOT NULL,
  category     VARCHAR(50)  NOT NULL,
  venue        VARCHAR(200) NOT NULL,
  event_date   DATETIME     NOT NULL,
  registration_deadline DATETIME NOT NULL,
  capacity     INT NOT NULL,
  fee          INT NOT NULL DEFAULT 0,
  team_min     INT NOT NULL DEFAULT 1,
  team_max     INT NOT NULL DEFAULT 1,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (organizer_id) REFERENCES users (id)
    ON DELETE CASCADE,
  CHECK (capacity > 0), CHECK (fee >= 0),
  CHECK (team_min >= 1 AND team_max >= team_min)
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS teams (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT          NOT NULL,
  name       VARCHAR(100) NOT NULL,
  join_code  VARCHAR(10)  NOT NULL UNIQUE,
  leader_id  INT          NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (event_id, name),
  FOREIGN KEY (event_id)  REFERENCES events (id)
    ON DELETE CASCADE,
  FOREIGN KEY (leader_id) REFERENCES users (id)
    ON DELETE CASCADE
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS team_members (
  team_id   INT NOT NULL,
  user_id   INT NOT NULL,
  joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (team_id, user_id),
  FOREIGN KEY (team_id) REFERENCES teams (id)
    ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users (id)
    ON DELETE CASCADE
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS registrations (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  user_id       INT NOT NULL,
  event_id      INT NOT NULL,
  team_id       INT NULL,
  status ENUM('confirmed','cancelled') NOT NULL
                DEFAULT 'confirmed',
  checked_in_at DATETIME NULL,
  registered_at TIMESTAMP NOT NULL
                DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, event_id),
  FOREIGN KEY (user_id)  REFERENCES users (id)
    ON DELETE CASCADE,
  FOREIGN KEY (event_id) REFERENCES events (id)
    ON DELETE CASCADE,
  FOREIGN KEY (team_id)  REFERENCES teams (id)
    ON DELETE SET NULL
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS announcements (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT          NOT NULL,
  title      VARCHAR(150) NOT NULL,
  content    TEXT         NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (event_id) REFERENCES events (id)
    ON DELETE CASCADE
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS event_questions (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT  NOT NULL,
  user_id    INT  NOT NULL,
  question   TEXT NOT NULL,
  answer     TEXT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (event_id) REFERENCES events (id)
    ON DELETE CASCADE,
  FOREIGN KEY (user_id)  REFERENCES users (id)
    ON DELETE CASCADE
) ENGINE = InnoDB;
CREATE TABLE IF NOT EXISTS event_winners (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  event_id    INT          NOT NULL,
  position    INT          NOT NULL,
  winner_name VARCHAR(150) NOT NULL,
  prize       VARCHAR(100) NULL,
  FOREIGN KEY (event_id) REFERENCES events (id)
    ON DELETE CASCADE,
  CHECK (position >= 1)
) ENGINE = InnoDB;
```

TABLE V
SUMMARY OF DECLARATIVE CONSTRAINTS

Constraint	No.	Examples
PRIMARY KEY	8	7 surrogate keys; composite (team_id, user_id)
FOREIGN KEY	12	11 CASCADE, 1 SET NULL
UNIQUE	4	email, join_code, (event_id, name), (user_id, event_id)
CHECK	4	capacity > 0, fee ≥ 0, 1 ≤ team_min ≤ team_max, position ≥ 1
ENUM / DEFAULT	2 / 11	role, status; timestamps, fee, team sizes

VII. DATA POPULATION (DML)

`sql/seed.sql` loads 8 users (3 organizers from VIT Vellore, VIT Chennai and IIT Madras, and 5 students), 7 events (one already completed), 3 teams with 6 memberships, 9 registrations (3 checked in, 1 cancelled), 3 announcements, 3 questions (1 unanswered) and 4 winners. Ids are explicit so foreign keys are easy to follow, and event dates are relative to `CURDATE()` so the demonstration always has open events. Only bcrypt hashes of the demo password are stored.

Listing 2. Representative DML (seed and run-time statements).

```
INSERT INTO users (id, name, email, password_hash,
  role, college, reg_no) VALUES
 (1,'Code Cell','codecell@showup.local','$2b$10$…',
  'organizer','VIT Vellore',NULL),
 (4,'Ananya Sharma','ananya@showup.local','$2b$10$…',
  'student','VIT Vellore','25BCE0101');
INSERT INTO teams (id, event_id, name, join_code,
  leader_id) VALUES (1, 1, 'NeuralCraft', 'NC8821', 4);
INSERT INTO team_members (team_id, user_id)
  VALUES (1, 4), (1, 5);
-- cancel: keep the row as history
UPDATE registrations SET status = 'cancelled',
  team_id = NULL WHERE id = ?;
-- check-in: idempotent, guarded by WHERE
UPDATE registrations SET checked_in_at = NOW()
 WHERE id = ? AND status = 'confirmed'
   AND checked_in_at IS NULL;
-- leader leaves: earliest member succeeds
UPDATE teams SET leader_id =
  (SELECT user_id FROM team_members WHERE team_id = ?
    ORDER BY joined_at, user_id LIMIT 1)
 WHERE id = ? AND leader_id = ?;
-- delete event: cascades to six child tables
DELETE FROM events WHERE id = ?;
```

Listings 3 and 4 show retrieval queries with their actual output on the seeded MySQL 8.0.46 database. In Listing 3 the LEFT JOIN keeps events that have no registrations (count 0), and the status filter is placed in the ON clause for the same reason; Listing 4 uses the fact that `COUNT(column)` ignores NULLs to obtain confirmed and checked-in totals in one pass.

Listing 3. Seats taken per event — INNER JOIN, LEFT JOIN, COUNT, GROUP BY (the query behind GET /api/events).

```
SELECT e.id, e.title, u.name AS organizer, e.capacity, COUNT(r.id) AS registered
  FROM events e JOIN users u ON u.id = e.organizer_id
  LEFT JOIN registrations r ON r.event_id = e.id AND r.status = 'confirmed'
 GROUP BY e.id ORDER BY e.event_date;
+----+---------------------------+-------------------------+----------+------------+
| id | title                     | organizer               | capacity | registered |
+----+---------------------------+-------------------------+----------+------------+
|  7 | Hack Night 2026           | Code Cell               |       30 |          3 |
|  2 | Figma to Front-End        | Code Cell               |       60 |          2 |
|  1 | CodeSprint 24             | Code Cell               |       40 |          2 |
|  5 | Founders' Fireside        | Code Cell               |      120 |          0 |
|  3 | Valorant 5v5 Showdown     | Cultural & Esports Club |       70 |          0 |
|  4 | Battle of the Bands       | Cultural & Esports Club |       48 |          0 |
|  6 | Line Follower Championship | Robotics Club           |       30 |          1 |
+----+---------------------------+-------------------------+----------+------------+
```

Listing 4. Attendance per event — COUNT(*) vs. COUNT(column).

```
SELECT e.title, COUNT(*) AS confirmed,
       COUNT(r.checked_in_at) AS checked_in
  FROM registrations r JOIN events e ON e.id = r.event_id
 WHERE r.status = 'confirmed' GROUP BY e.id, e.title;
+---------------------------+-----------+------------+
| title                     | confirmed | checked_in |
+---------------------------+-----------+------------+
| CodeSprint 24             |         2 |          0 |
| Figma to Front-End        |         2 |          0 |
| Line Follower Championship |         1 |          0 |
| Hack Night 2026           |         3 |          3 |
+---------------------------+-----------+------------+
```

VIII. Front-End Design

The interface is a single-page application in React 19 and TypeScript [9], bundled by Vite and styled with Tailwind CSS. Fig. 1 shows the three-tier architecture: the browser never connects to MySQL and obtains all data through the typed wrapper `src/lib/api.ts`, which attaches the user's token to each request.

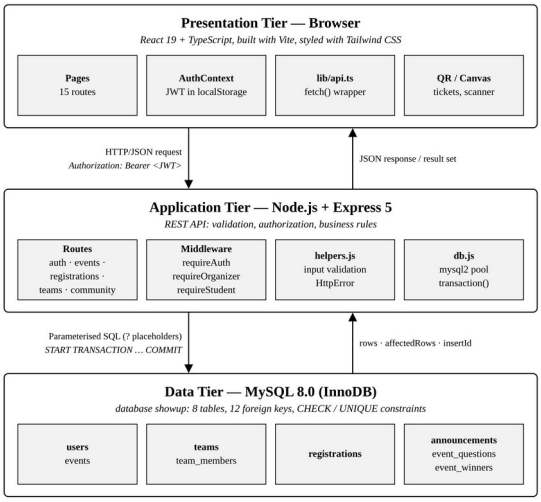
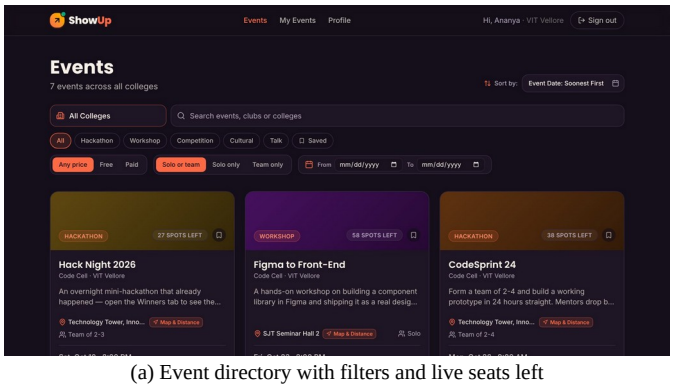
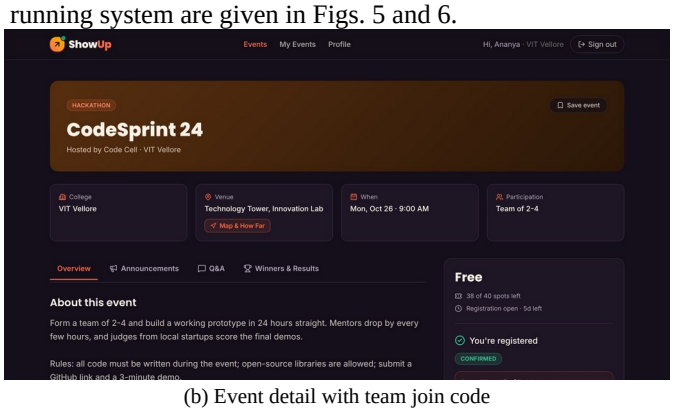


Fig. 1. Three-tier architecture: React presentation tier, Express application tier and MySQL 8 data tier.

Students use the landing, sign-up/log-in, event directory (search, filters, live seats left), event detail (overview, announcements, Q&A and winners tabs, register / create team / join team), My Events (QR tickets, team, cancel) and profile pages. Organizers use the Organizer Studio (own events and fill rate), create/edit event form, participants page (grouped by team, CSV export) and QR check-in page. Navigation is role-aware, values such as seats left are recomputed by SQL on every load, and server error messages (e.g. "Sorry, this event is full.") are shown to the user verbatim. Screenshots of the running system are given in Figs. 5 and 6.

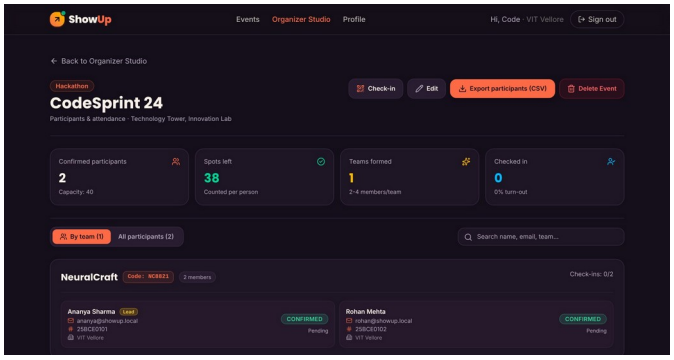


(a) Event directory with filters and live seats left

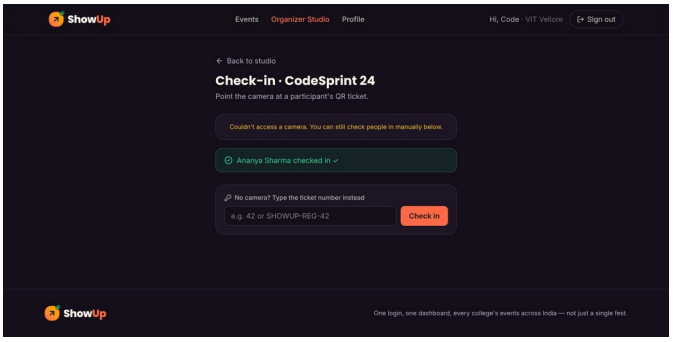


(b) Event detail with team join code

Fig. 5. Student pages. "Spots left" on each card is capacity minus the COUNT of confirmed registrations computed in SQL.



(a) Participants grouped by team



(b) QR check-in (manual entry)

Fig. 6. Organizer pages. A successful check-in sets registrations.checked_in_at.

IX. Business Logic Implementation

Rules expressible declaratively are placed in the schema so they hold for any client; rules that depend on time, aggregates or the requester's identity are implemented in Express route handlers inside transactions. Where possible both layers apply: the application gives a clear message and the constraint is the backstop.

TABLE VI
Business Rules and Enforcement

ID	Rule	Enforced by
BR-1	Unique e-mail per account	UNIQUE + API
BR-2	capacity > 0, fee ≥ 0, 1 ≤ team_min ≤ team_max	CHECK + API
BR-3	No registration after the deadline	API (transaction)
BR-4	One registration per student per event	UNIQUE + API
BR-5	Confirmed registrations ≤ capacity	Row lock + COUNT
BR-6	Team size ≤ team_max when joining	API: COUNT
BR-7	Only students register; only organizers create events	Middleware
BR-8	Only the owner edits, deletes, checks in	Ownership check
BR-9	A ticket is checked in at most once	Guarded UPDATE
BR-10	Leader leaves → earliest member leads	API (transaction)

Registration transaction (Fig. 7): the handler starts a transaction and executes `SELECT … FROM events WHERE id = ? FOR UPDATE`. InnoDB grants an exclusive row lock held until COMMIT or ROLLBACK, so concurrent registrations for the same event are serialized and the capacity check followed by INSERT is atomic [14]; registrations for different events lock different rows and run in parallel. Any failed check throws and triggers ROLLBACK.

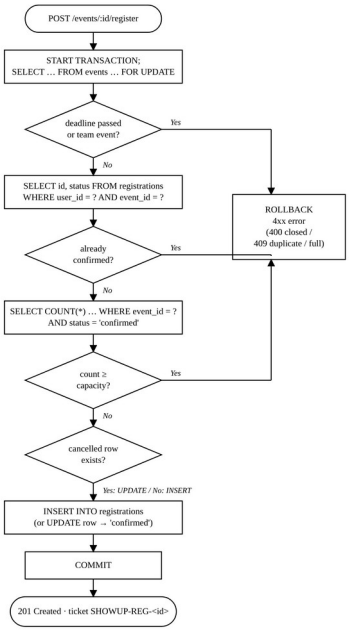


Fig. 7. Solo-registration transaction (POST /api/events/:id/register).

Teams: creating a team locks the event, checks the deadline, duplicates and capacity, then inserts into `teams`, `team_members` and `registrations` in one transaction. The join code has six characters from a 32-symbol alphabet without look-alikes (0/O, 1/I), giving $32^6 \approx 1.07 \times 10^9$ codes, generated with `crypto.randomInt`. Joining finds the team by code, locks its event, and checks `COUNT(*)` of members against `team_max` and seats against capacity.

Cancellation: the status becomes `'cancelled'` (the seat is freed but history is kept; re-registering reactivates the same row and ticket). A team member's `team_members` row is deleted; an empty team is deleted, and if the leader left, the earliest member becomes leader via the correlated sub-query in Listing 2.

TABLE VII
Authorization Matrix

Action	Visitor	Student	Organizer
Browse events, Q&A, winners	✓	✓	✓
Register, create / join team		✓	
Cancel registration		own	own event
Create event			✓
Edit, delete, check-in, announce, answer			own event

Middleware `requireAuth` verifies the token; `requireStudent` and `requireOrganizer` enforce roles; ownership is checked against `events.organizer_id`. Input helpers trim text, bound lengths and ranges, and reject invalid categories or dates with HTTP 400.

X. Database Connectivity

The Express tier [8] connects through the *mysql2* driver [13] with a pool of ten connections configured from a git-ignored `.env` file (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`). Each connection sets `time_zone = '+00:00'` so date-times are stored in UTC. All statements of a transaction must share one connection, which the helper in Listing 5 checks out and always releases.

Listing 5. Connection pool and transaction helper (server/db.js).

```
export const pool = mysql.createPool({
  host, port, user, password, database,
  connectionLimit: 10, timezone: "Z" });

export async function transaction(work) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await work(conn);
    await conn.commit();
    return result;
  } catch (err) {
    await conn.rollback(); throw err;
  } finally { conn.release(); }
}
```

No SQL string is built by concatenating user input: values are bound to `?` placeholders by the driver, the primary defence recommended by OWASP [12]. Passwords are hashed with bcrypt (cost 10) [11]; log-in returns a JSON Web Token [10] with the user's id and role, valid for seven days, so no sessions table is needed. The REST API exposes 23 endpoints; for example `GET /api/events` runs the query of Listing 3, and `POST /api/teams/join` runs the transaction of Section IX. A central error handler maps MySQL `ER_DUP_ENTRY` (1062) to HTTP 409, connection errors to 503, and hides internal details behind 500.

XI. Testing and Validation

Testing followed IEEE Std 829 [15]: every case has an identifier, input, expected and actual result, and verdict. Four levels were used: (i) *static checks*—TypeScript compilation and the production build succeed and the linter reports 0 errors; (ii) *API black-box tests*—a script sends real HTTP requests to the running server; (iii) *database constraint tests*—invalid SQL is executed directly against MySQL to show the DBMS rejects it independently of the application; and (iv) a *manual UI walkthrough* of every page. Tests ran on MySQL 8.0.46 and Node.js 22 starting from freshly seeded data. Of 35 cases, all passed; Table VIII lists a representative subset.

TABLE VIII
Representative Test Cases

ID	Test	Expected	Actual	Result
TC-01	Login, valid credentials	200 + JWT	200	Pass
TC-02	Login, wrong password	401	401	Pass
TC-04	Sign up, existing e-mail	409	409	Pass
TC-07	Solo registration, open event	201	201	Pass
TC-08	Duplicate registration	409	409	Pass
TC-09	Register after deadline	400	400	Pass
TC-12	Re-register after cancel	201, row 9 reused	201, id 9	Pass
TC-13	Duplicate team name	409	409	Pass
TC-16	Join team with code	201	201	Pass
TC-18	Join a full team (4/4)	409	409	Pass
TC-19	Check in valid ticket	200	200	Pass
TC-20	Check in same ticket twice	409	409	Pass
TC-21	Check-in by other organizer	404	404	Pass
TC-26	5 concurrent requests, 1 seat	1×201, 4×409	1×201, 4×409	Pass
TC-27	Leader cancels	leader → id 5	leader_id = 5	Pass
TC-28	SQL: capacity = 0	CHECK error	3819	Pass
TC-30	SQL: event_id = 999	FK error	1452	Pass
TC-31	SQL: duplicate (user, event)	UNIQUE error	1062	Pass
TC-34	Delete event	children removed	9 rows → 0	Pass
TC-35	Login with ' OR '1'='1	401	401	Pass

TABLE IX
Test Summary (All 35 Cases)

Module	Cases	Passed
Auth	6	6
Registration	6	6
Teams	7	7
Check-in	3	3
Events	4	4
Concurrency	1	1
DB constraint	7	7
Security	1	1
Total	**35**	**35**

Concurrency: in TC-26 an event with capacity 1 received five registration requests simultaneously; exactly one succeeded and four received "Sorry, this event is full." Without the `FOR UPDATE` lock all five transactions could read a count of 0 before any inserted—the lost-update anomaly.

Integrity: the DBMS itself rejected a CHECK violation (error 3819), a dangling foreign key (1452), a duplicate registration (1062), an out-of-domain ENUM value (1265) and a NULL title (1048). Deleting an event removed its team, 3 registrations, announcement and 4 winners through ON DELETE CASCADE, and when a team leader cancelled, leadership passed to the next member.

XII. Project Demonstration and Documentation

The system is installed with `npm install`, configured by copying `.env.example` to `.env`, initialised with `npm run db:reset` (runs `schema.sql` and `seed.sql`; refused unless the host is local) and started with `npm run dev`. The same

SQL files can be run from MySQL Workbench. Seed accounts (password *password123*) include the organizer *codecell@showup.local* and students such as *sameer@showup.local*. The demonstration shows each action's effect in MySQL Workbench:

1) `SHOW TABLES` and `DESCRIBE registrations` present the schema.
2) Sameer joins team NC8821 for CodeSprint 24; new rows appear in `team_members` and `registrations`; a second attempt shows the 409 duplicate error.
3) Code Cell opens *Participants*, then *Check-in*, enters Sameer's ticket, and `checked_in_at` is set.
4) The queries of Listings 3–4 are run, and a test event is deleted to show ON DELETE CASCADE.

Documentation comprises the README (features, setup, database design, every foreign key, example queries, API table), the commented SQL scripts, inline comments explaining the SQL of each route, this report, and the Git history recording the evolution from an earlier 12-table design to the present 8-table schema.

XIII. Conclusion and Future Work

ShowUp applies the complete database development life cycle to a real problem: requirements were modelled as an ER/EER diagram, mapped to eight relations proven to be in BCNF, and implemented in MySQL with twelve foreign keys and CHECK, UNIQUE and ENUM constraints, while time- and aggregate-dependent rules run in ACID transactions with row-level locking. All 35 tests passed, including the concurrency test. Future work includes a waitlist promoted by a trigger or stored procedure, views for organizer reports, an index on `events.event_date` and a FULLTEXT index for search, and an audit table.

Acknowledgment

The author sincerely thanks Prof. Pooja Chaudhary, faculty for the Database Management Systems course at Vellore Institute of Technology, for the guidance and support provided throughout this project.

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[3] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[4] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976.

[5] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[6] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[7] Oracle Corporation, "MySQL 8.0 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] OpenJS Foundation, "Express 5.x API reference." [Online]. Available: https://expressjs.com/en/5x/api.html

[9] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[11] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999, pp. 81–91.

[12] OWASP Foundation, "SQL Injection Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/

[13] A. Sidorov *et al.*, "node-mysql2." [Online]. Available: https://github.com/sidorares/node-mysql2

[14] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[15] *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008, 2008.